

Avoiding errors and bugs

Whenever we write some code, we need to check that it's correct. Since Java is a compiled language, the compiler does some of these checks — for example:

- You might write the wrong name for a variable, method or class.
- You might use the wrong type of value in the wrong place.
- You might give too few or too many arguments to a method.
- You might forget to write a return statement at the end of a method.
- You might use a variable in the wrong place, where it isn't defined.
- You might forget a semicolon or some other minor syntax detail.

Every programmer makes these mistakes sometimes. Fortunately, the compiler detects all of them, so they're easy to discover and fix. These mistakes are called “**compilation errors**” or “**compile-time errors**”.

On the other hand, some mistakes are more difficult to find — if your code compiles without errors, but it does the wrong thing when it runs, then you won't find out until you test it. These mistakes are called “**bugs**”, and almost every piece of software has some bugs. There are lots of different kinds of bugs, but some common reasons for bugs are:

- Writing the wrong logical condition — e.g. `i <= n` instead of `i < n`.
- Not checking for invalid inputs.

Using the wrong standard library function — e.g. `Integer.getInteger` when you should have used `Integer.parseInt`. The `getInteger` method actually determines the integer value of the system property with the specified name ([check class library specification](#)).

Being a good programmer does **not** mean you have to write code that always works correctly on the first attempt. Rather, a good programmer can:

- **Discover** that there is a bug,
- **Isolate** the code causing the bug,
- **Diagnose** why the isolated code causes the bug,
- **Fix** the bug, and
- **Check** that the bug has really been fixed.

This is called “**debugging**”. Testing helps with this process.

Manual testing

The simplest way to test code is “by hand”; that is, by running the code to see whether it does the right thing. Consider the following example:

```
class Vector {  
    int x;  
    int y;  
  
    Vector(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    int getX() {  
        return x;  
    }  
  
    int getY() {  
        return y;  
    }  
  
    double length() {  
        return Math.sqrt(x^2 + y^2);  
    }  
}
```

This class compiles without errors. Using an IDE (Integrated Development Environment), we can test whether it behaves as we expect:

```
Vector vector = new Vector(3, 4);  
vector.getX()  
    3 (int)  
vector.getY()  
    4 (int)  
vector.length()  
    2.6457513110645907 (double)
```

The `getX` and `getY` methods do what we expect, but the correct length of this vector should be 5.0, which is not the result we actually got — so we have **discovered** a bug.

Using the numbers we’ve chosen, the formula $3^2 + 4^2$ should give 25, then the result of `Math.sqrt` should be 5. The most likely cause of the bug is in the code `x^2 + y^2`, so we can try this out with the values 3 and 4:

```
3^2 + 4^2  
    7 (int)
```

This is not the expected result, so we have **isolated** this code as the cause of the bug.

The expected behaviour of this code is that 3^2 should be 9, then 4^2 should be 16, then $9 + 16$ should be 25. However, using the Code Pad we can see that the $\wedge$ operator does not do what we expect:

```
3^2      1 (int)
4^2      6 (int)
```

Now we have **diagnosed** the bug — it is caused by an incorrect use of the $\wedge$ operator. It's actually a “[bitwise exclusive or](#)”, but it's easy to mistake it for “[to the power of](#)”.

The code must be changed so that it correctly squares x and y . The simplest solution is to write $x*x$ instead of x^2 (we can also use the `pow` method of the `Math` class):

```
double length() {
    return Math.sqrt(x*x + y*y);
}
```

After compiling the class again, we now believe we have **fixed** the bug. To be sure, we should **check** that the method now works correctly. We can use the IDE again for this:

```
Vector vector = new Vector(3, 4);
vector.length()
    5.0 (double)
```

Now we get the expected output, so we have some confidence that this bug is fixed. However, there may still be other bugs; more testing will be required. In fact, there is another bug in this method, but it is very subtle. Can you find it?

Automated testing

Fixing a bug — and checking that it's fixed — is not the end of the story. In real software, bugs often appear in code because of later changes, even changes in other classes; these are called **regression bugs**. We don't want to have to go through the same manual testing process every time the Vector class changes; manual testing is fairly easy, but it's slow, and we might make mistakes.

Anything easy and repetitive should be done by the computer instead. Let's write a program which does our testing automatically:

```
class VectorTest {
    static void main() {
        Vector vector = new Vector(3, 4);

        int x = vector.getX();
        System.out.print("Testing getX() method: ");
        if(x == 3) {
            System.out.println("passed");
        } else {
            System.out.println("failed, expected 3, was " + x);
        }

        int y = vector.getY();
        System.out.print("Testing getY() method: ");
        if(y == 4) {
            System.out.println("passed");
        } else {
            System.out.println("failed, expected 4, was " + y);
        }

        double length = vector.length();
        System.out.print("Testing length() method: ");
        if(length == 5.0) {
            System.out.println("passed");
        } else {
            System.out.println("failed, expected 5.0, was " + length);
        }
    }
}
```

The VectorTest program runs with the following output, showing us that all three tests gave the expected results:

```
Testing getX() method: passed
Testing getY() method: passed
Testing length() method: passed
```

However, if we run the VectorTest program with the original version of the Vector class, it detects the bug:

```
Testing getX() method: passed  
Testing getY() method: passed  
Testing length() method: failed, expected 5.0, was 2.6457513110645907
```

So automated testing is useful for **discovering** bugs and **checking** that they've been fixed.